

Elevator Prediction System Design

1. The Approach: A Simulation-First Mindset

1. Model the Core Components:

- The `<Elevator>` object will track its own state: current floor, direction of movement, and its queue of destination floors.
- The `<ElevatorSystem>` will manage the elevator, handle incoming calls (*demands*), and, most importantly, **be responsible for logging the data** we need.

2. Identify the “Golden event” for Data Logging:

The most critical data point for the future ML model is the link between a **resting state** and the **next call**. Therefore, the key event to log is: “An elevator was resting on floor **X** at time **T**, and the very next call came from floor **Y**.”

3. Create a Data-Centric API:

The system will be controlled via a simple **REST API**. This API will allow us to simulate demand (`/call`) and observe the system (`/status`). The API endpoint that receives the call is the perfect place to trigger our data logging logic.

4. Choose a Scalable Storage Solution:

The data needs to be stored durably. I’ll start with a simple file-based database (SQLite) for ease of setup, but the design will use an ORM (Object-Relational Mapper) like SQLAlchemy, making it trivial to switch to a production-grade database like PostgreSQL later.

5. Implement a Discrete Time Simulation:

To make the simulation predictable and testable, I will not use real-time threads. Instead, I’ll include an endpoint (`/step`) that advances the simulation by one “tick” or time unit. In each tick, the elevator will move one floor or open/close its doors.

2. Data Modelling: What to Collect and Why

The goal is to predict the **next_call_floor**.

An ML model learns a function $f(\text{features}) \rightarrow \text{label}$. We need to define our features and our label.

- Label (What we want to predict):** **call_floor**. The floor from which the next demand originates.
- Features (The context we use for prediction):**
 - resting_floor**: The floor the elevator was on when it was idle. **This is the most important feature.**
 - timestamp_rested**: The date and time the elevator became idle. This is crucial for identifying temporal patterns.
 - day_of_week**: Derived from the timestamp. Demand on a Monday morning is very different from a Saturday night.
 - hour_of_day**: Derived from the timestamp. Captures rush hours, lunchtimes, etc.

Based on this, I will create a single table in our database, let’s call it **demand_log**. Each row in this table will represent a single “**call event**” that occurred after a resting period.

Column Name	Data type	Description	Importance for ML
-------------	-----------	-------------	-------------------

id	Integer	Primary Key	N/A (System use)
timestamp_rested	DateTime	When the elevator became idle.	High. Captures time-based patterns.
timestamp_called	DateTime	When the next call was received.	High. Used to calculate idle time and for sequencing.
resting_floor	Integer	The floor the elevator was resting on.	Critical Feature. The primary input for our model.
call_floor	Integer	The floor the call came from.	Critical Label. The value we want to predict.
destination_floor	Integer	Where the user wants to go.	Medium. Could be a feature for more advanced models.
day_of_week	Integer	0-6 (Monday-Sunday)	High. A direct feature for the model to learn weekly patterns.
hour_of_day	Integer	0-23	High. A direct feature for the model to learn daily patterns.